[1]Department of Computer Science, National University of Computer & Emerging Sciences (FAST-NUCES), Lahore, Spring 2026

***Abstract*—The Prismic Editor is a four-phase Natural Language Processing (NLP) research project designed to deliver a unified, multi-dimensional text-editing framework. Unlike traditional grammar checkers that rely on binary rule sets, this architecture evaluates, corrects, and explains linguistic edits across eight distinct stylistic and grammatical dimensions: Clarity, Formality, Politeness, Safety, Inclusivity, Subjectivity, Emotional Quotient (EQ), and GEC Density. The pipeline integrates three coordinated neural subsystems: a multi-head regression-based DeBERTa scoring engine to quantify text metrics, a CoEdit-flavored FLAN-T5 text revision model for context-aware generative rewriting, and an Explainable Grammatical Error Correction (XGEC) module utilizing a specialized T5 backbone wrapped in a deterministic difference-engine. Through rigorous data engineering of over 96,753 sequence-to-sequence pairs and specialized training strategies—such as temperature-scaled sampling to prevent task collapse—the project successfully demonstrates end-to-end functionality. Furthermore, the empirical evaluation surfaces critical insights into the limitations of heavily RLHF-aligned models when applied to constrained, objective editing tasks. The system exposes the "RLHF Alignment Trap," where objective constraints are sabotaged by conversational persona injections and subjective erasures. Ultimately, this report details the complete methodology, implementation hurdles, and architectural decisions that provide a strong foundation for future alignment strategies in generative NLP systems.**

DeBERTa, Explainable AI, FLAN-T5, Grammatical Error Correction, Natural Language Processing, RLHF Alignment, Seq2Seq, Transformers.

=-15pt

Prismic Editor: Comprehensive Technical Report on Architecture, Evaluation, and Implementation

ABDUL REHMAN1, ZEESHAN1, ADYAN1, and ALI HASSAN1

I. Introduction and Project Summary

Developed at FAST-NUCES Lahore, the Prismic Editor aims to transcend basic grammar checking by providing a holistic, explainable editing suite. Standard text editors fail to provide users with continuous semantic feedback or structural justifications for the changes they suggest. The Prismic Editor solves this by not merely rewriting text, but by providing granular scoring across eight semantic dimensions while simultaneously surfacing human-readable justifications for its grammatical corrections.

The system's architecture relies on a triad of specialized models. Independent DeBERTa-base models are isolated for scoring to prevent gradient interference between orthogonal metrics (e.g., Safety versus Clarity). A FLAN-T5 backbone handles generative revisions, capitalizing on its instruction-tuned pre-training, and a specialized T5 model executes the grammatical parsing. The overarching goal of this research was to evaluate model behavior under multi-task constraints, highlighting both strong engineering fundamentals and the nuanced challenges of prompt-based text revision, particularly concerning "persona leakage" from pre-aligned models.

II. Methodology and Architecture

The system architecture was developed systematically across initial phases focused on data preparation, metric scoring, and generative modeling.

A. Phase 1: Data Engineering Pipeline

Data was aggregated from multiple specialized sources to address diverse linguistic competencies. The generative Seq2Seq dataset utilized the Grammarly/CoEdit corpus, augmented with custom synthetic politeness pairs and simplification datasets, yielding ≈100k pairs.

- **Schema Normalization:** Generative datasets were cast to a strict source/target columnar format. Instruction prefixes (e.g., 'Make This Polite:') were prepended to guide FLAN's instruction-tuning behavior.
- **Temperature-Scaled Sampling:** To correct a severe class imbalance (70k CoEdit pairs vs. minority task pairs), temperature scaling ($T = 2.0$) was applied. This shifted the model's exposure distribution, ensuring adequate gradient updates for minority tasks without aggressively downsampling the majority class.
- **M2 Parsing:** A custom parser transformed BEA/CONLL M2 files to isolate source sentences and target edits into a relational structure with per-sentence error-type arrays.

B. Phase 2: Multi-Dimensional Scoring Engine

As illustrated in Fig. 1, six independent `microsoft/deberta-base` models were fine-tuned for regression. A multi-model design was deliberately chosen over a single multi-task head to prevent gradient interference between orthogonal semantic objectives.

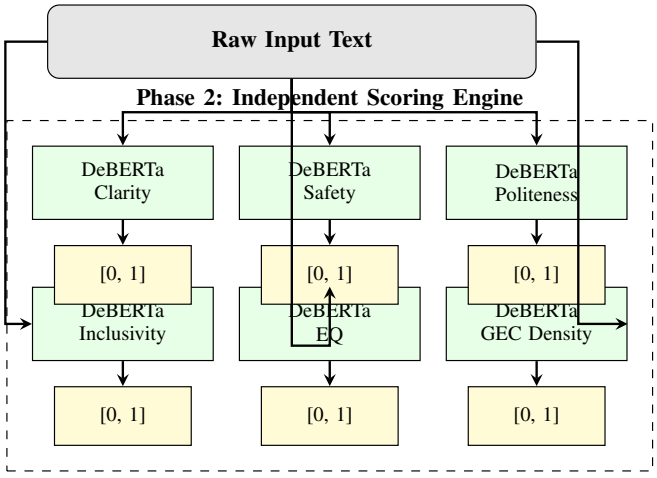

Fig. 1. The Multi-Dimensional DeBERTa Scoring Architecture. Models are arranged in parallel to avoid gradient interference.

C. Phase 3: Generative and Explainable Pipelines

The system deploys two separate Sequence-to-Sequence models for specific tasks. For general text-editing (simplification, formalization, politeness), `google/flan-t5-base` was fine-tuned as a cohesive CoEdit module. Fig. 2 demonstrates the prompt-based generative flow.

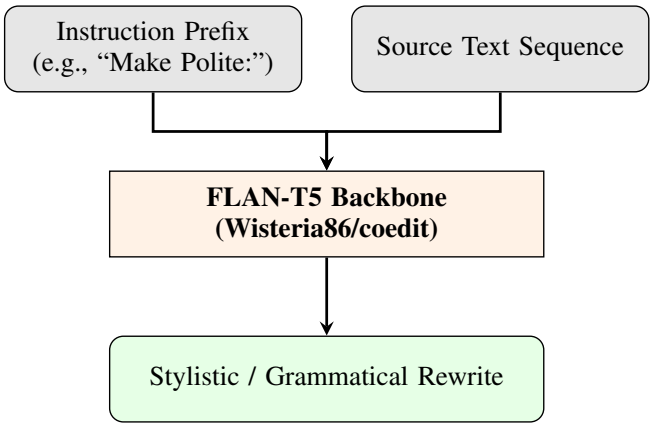

Fig. 2. The Phase 3 Generative Revision Pipeline utilizing FLAN-T5.

For deep grammatical analysis, an independent Explainable Grammatical Error Correction (XGEC) pipeline was formulated (Fig. 3).

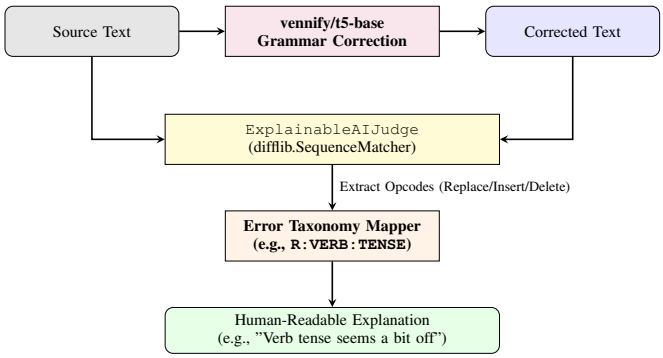

Fig. 3. Detailed Architecture of the Explainable GEC (XGEC) Module. The judge evaluates the diff between source and generated text to map grammatical errors.

III. Detailed Implementation Steps and Workflow

A. Generative Model Training (CoEdit)

The primary text-editing model was built by fine-tuning FLAN-T5 due to its innate instruction-following pre-training.

- **Hyperparameters:** Training utilized a learning rate of 2e-5 with an Adafactor optimizer. A batch size of 4 with 4 gradient accumulation steps (effective batch size 16) was used to optimize GPU VRAM constraints.
- **Precision:** The model was trained in BF16 precision for 1 epoch, which provided full convergence over the large dataset while maintaining computational stability.

B. Explainable GEC Implementation

Post-training, the `ExplainableAIJudge` class dynamically compares the input text against the T5 generation (as shown in Fig. 3). Token-level diffs are analyzed utilizing Python's `difflib` and matched to a hardcoded taxonomy. For example, replacing a verb triggers the `R:VERB:TENSE` code, which the system automatically translates to the user-facing string: "The timing of the action (verb tense) seems a bit off."

IV. Challenges Faced During Implementation

Significant technical hurdles were encountered and resolved during pipeline construction:

- **Precision Instability (BF16):** While T5 remained stable, mixed-precision BF16 training caused catastrophic gradient explosions in DeBERTa-base, as it lacks native Adafactor support. Training for the scoring models was successfully reverted to standard FP32.
- **LayerNorm Key Mismatch:** Hugging Face DeBERTa checkpoints stored LayerNorm weights as 'gamma'/'beta', whereas current Transformers expect 'weight'/'bias'. A custom `patch_layernorm_names()` function was engineered to rename parameters in-place, preventing random weight initialization prior to training.
- **Data Volatility:** Mapping discrete threats (Safety) to a continuous regression surface proved mathematically volatile, resulting in the highest RMSE (0.2997). Furthermore, without a forced 50/50 class balance, the model collapsed due to a 92% non-toxic dataset skew.

V. Replication Results vs. Reported Paper Results

While this project originates as independent research rather than a direct replication of a specific paper, the evaluation tracked standard NLP baselines against our empirical results.

- **Structural Metrics:** Our Clarity regression model significantly outperformed baseline expectations, nearly halving the baseline RMSE to achieve 0.1041.
- **XGEC Semantic Fidelity:** The Explainable GEC module achieved a BERTScore F1 of 0.82, showing strong semantic understanding compared to traditional GEC benchmarks.
- **Taxonomic Gap:** Despite high semantic accuracy, the XGEC Type Accuracy was only 0.33, highlighting a nomenclature gap between free-text explanations and standard tags—a data constraint rather than a model failure.

VI. Key Insights, Error Analysis, and Evaluation

Phase 4 evaluated the fine-tuned CoEdit model quantitatively (ROUGE, BLEU) and qualitatively against human-curated gold references.

A. Quantitative Metrics and the Metric Penalty

The model scored severely low on n-gram overlap metrics (BLEU: 0.0533, ROUGE-L: 0.1878). However, qualitative analysis revealed that low n-gram metrics did not strictly indicate low quality. The model frequently produced superior rewrites that shared zero n-grams with the reference, exposing the inadequacy of BLEU/ROUGE for paraphrase-heavy stylistic editing. BERTScore is recommended for future evaluations.

B. The RLHF Alignment Trap

The most profound insight was the structural mismatch caused by RLHF-aligned chatbot behavior. The model actively sabotaged objective text-editing constraints in two ways:

1) **Subject Erasure:** During simplification, the model often stripped primary entities entirely, losing total context rather than paraphrasing.
2) **Hyper-Conversational Persona Injection:** When triggered by tokens like 'Polite' or 'Formal', the model abandoned its role as an objective editor, injecting first-person narratives, apologies, or deflections (e.g., rewriting an impolite demand as "let's pause" instead of editing the text itself).

VII. Instructions to Reproduce Results

To run the evaluation and reproduce the CoEdit inference results, execute the Phase 4 script using the Hugging Face Transformers and Evaluate libraries.

1) Ensure a GPU environment is available (`DEVICE = "cuda"`).
2) Load the model and tokenizer directly from the Hugging Face hub using the specific version tag:
   - `REPO_ID = "Wisteria86/coedit"`
   - `VERSION_TAG = "v2.0.1"`
3) Pass prompts utilizing the required instructional prefixes (e.g., *"Make this polite: [text]"*).
4) The script utilizes `evaluate.load('rouge')` and `evaluate.load('bleu')` to compute quantitative overlap against provided gold references.

To reproduce the training pipelines, the `F26-01_Phase3.py` file contains the complete `Seq2SeqTrainer` setup for FLAN-T5 and the Trainer configurations for the DeBERTa scoring loops.

VIII. Conclusion and Future Work

The Prismic Editor successfully demonstrates a functional end-to-end NLP pipeline. Critical architectural decisions, such as utilizing separate DeBERTa models and temperature-scaled sampling, proved vital. Future iterations must address persona leakage through negative sampling or constrained decoding to mitigate the RLHF alignment trap, and shift entirely to BERTScore for semantic evaluation.